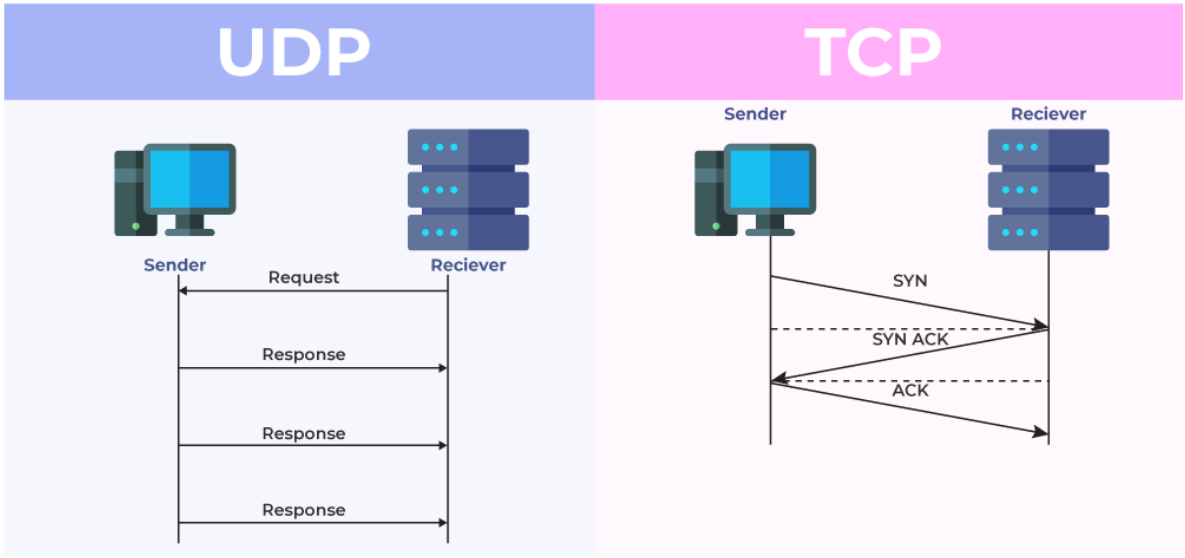
REPORTE DE PRÁCTICA 02: Concurrencia con hilos: teoría y práctica con TCP y UDP

Experiencia Educativa: Programación en Red

Alumno: Jorge Gael Hernández Marcial

Fecha: 08/10/2026



Objetivos de la sesión

• Comprender qué es un hilo y en qué se diferencia de un proceso.

• Programar concurrencia en Python con el módulo threading.

• Aplicar hilos en un servidor TCP para atender varios clientes de forma simultánea.

• Aplicar UDP para consultas ligeras sin establecer conexión.

• Sentar las bases de la Práctica 6: sistema de turnos LA FILA.

Marco Teórico / Fundamentos de Red

- Modelo Cliente-Servidor: Arquitectura de red donde un programa central (el servidor) provee recursos, datos o gestión de estado a múltiples programas periféricos (clientes) que realizan solicitudes independientes.
- Sockets: Punto final (endpoint) de comunicación en una red. Interfaz de software que permite a los procesos del sistema operativo enviar y recibir datos a través de los protocolos de la suite TCP/IP.
- Protocolo TCP (Transmission Control Protocol): Protocolo orientado a conexión que establece un enlace firme (Handshake) antes de transferir datos. Garantiza que la información llegue en orden, sin pérdidas ni duplicados. Es ideal para acciones críticas (como asignar un turno único).
- Protocolo UDP (User Datagram Protocol): Protocolo no orientado a conexión que envía datagramas de forma rápida y directa (Fire-and-forget) sin asegurar su llegada. Es ideal para consultas de solo lectura donde la velocidad prima sobre la confiabilidad (como preguntar cuántos turnos van).
- Concurrencia e Hilos (Multithreading): Capacidad de un programa para dividir su ejecución en múltiples hilos que operan en paralelo. Permite al servidor mantener la escucha en varios puertos (TCP y UDP) y atender a múltiples clientes sin que el código se bloquee en una sola tarea.
- Exclusión Mutua (Locks): Mecanismo de sincronización que protege variables compartidas en memoria (como el contador global de turnos). Asegura que solo un hilo a la vez pueda leer y modificar la variable, previniendo condiciones de carrera (Race conditions).

Desarrollo y Metodología

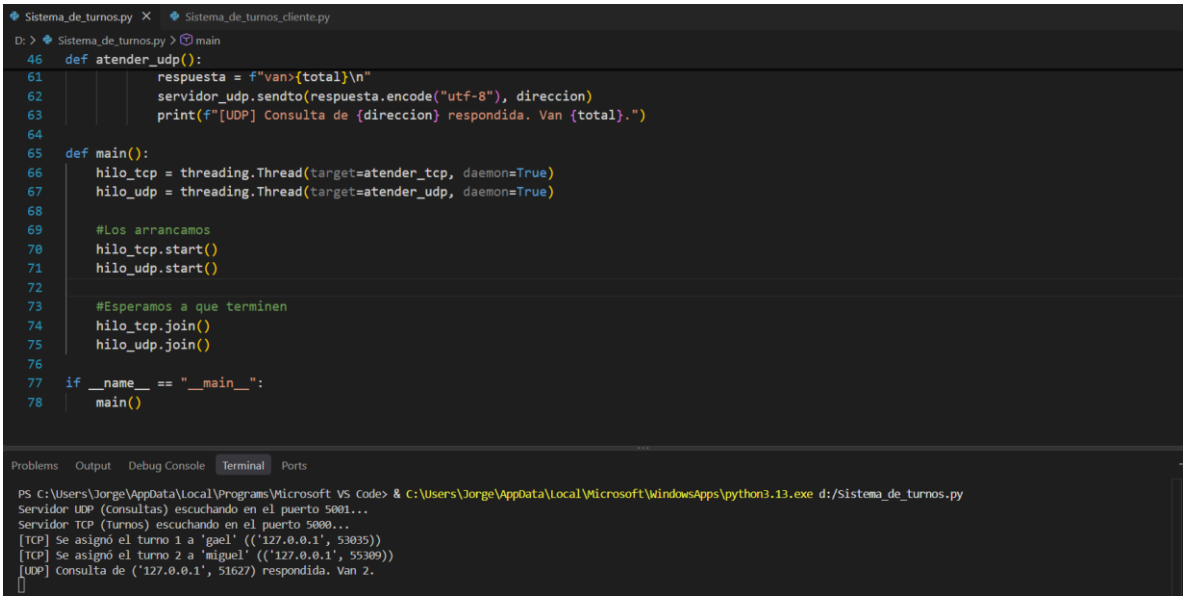
A. Implementación del Servidor (Sistema_de_turnos.py) El servidor opera como un sistema multihilo concurrente ejecutando las siguientes tareas en paralelo:

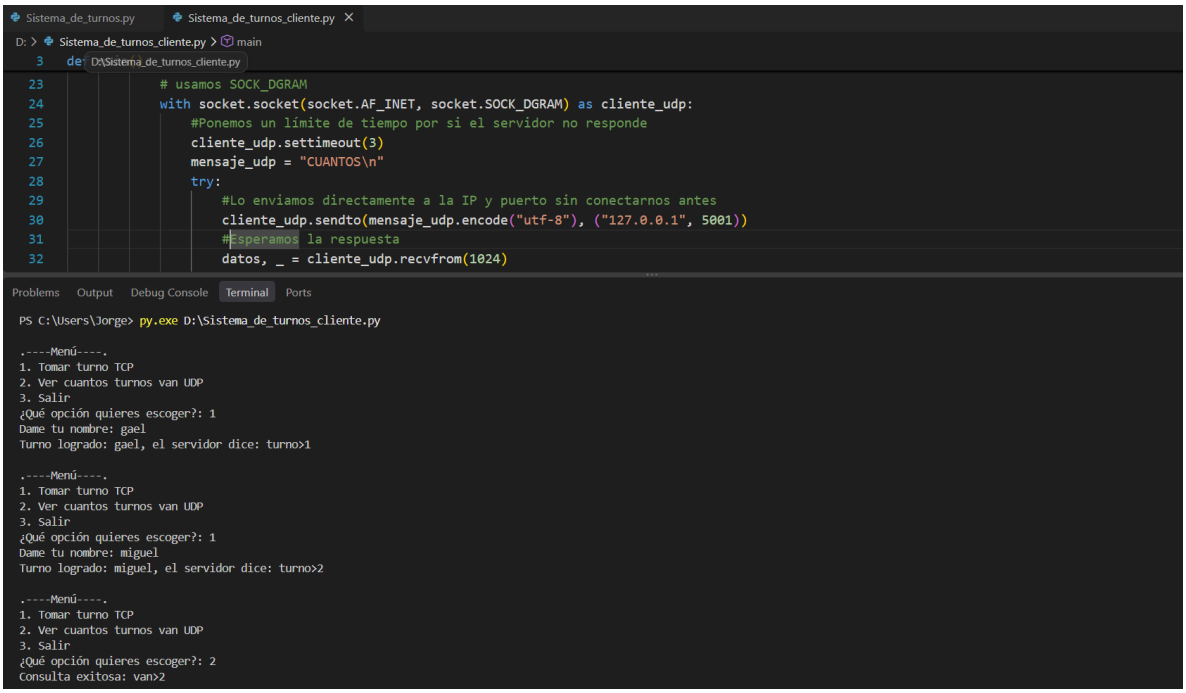
1. Hilos Principales de Escucha: El motor principal (main) arranca dos hilos independientes (Daemon Threads). Uno se encarga de escuchar el puerto TCP 5000 y el otro el puerto UDP 5001 simultáneamente.
2. Lógica TCP (Asignación de Turnos):
    - Configura el socket TCP (SOCK_STREAM) en el puerto 5000 (bind y listen).
    - En un bucle infinito, se bloquea en accept() esperando clientes.
    - Al conectarse un cliente, el servidor delega la conexión a un *nuevo hilo independiente* ejecutando la función tomar_turno.
    - El hilo procesa el mensaje (TURNO>apodo), adquiere el candado de seguridad (with candado_turno:), incrementa el contador global y responde al cliente de forma segura con el formato turno>N.
3. Lógica UDP (Consultas de Estado):
    - Configura el socket UDP (SOCK_DGRAM) en el puerto 5001.
    - Se bloquea en recvfrom() esperando datagramas.
    - Al recibir el mensaje CUANTOS, lee el contador global protegido por el candado y devuelve inmediatamente la cifra total (van>N) usando sendto().

B. Implementación del Cliente (Sistema_de_turnos_cliente.py) El cliente funciona como una interfaz interactiva de consola mediante un bucle de menú:

1. Flujo TCP (Opción 1): Utiliza socket.create_connection() para establecer un canal seguro con el puerto 5000. Solicita el nombre del usuario, formatea el mensaje (TURNO>apodo\n), lo envía, espera la asignación del turno, la imprime en pantalla y cierra la conexión ordenadamente.
2. Flujo UDP (Opción 2): Crea un socket UDP y configura un temporizador límite (settimeout(3)) para evitar bloqueos infinitos si el servidor no responde o el paquete se pierde. Envía el datagrama (CUANTOS\n) al puerto 5001 sin saludo previo, procesa la respuesta del servidor y la presenta al usuario.

4. Evidencias de Ejecución (Capturas de Pantalla)





Conclusiones

La práctica permitió comprobar de forma empírica la versatilidad de combinar distintos protocolos de transporte dentro de una misma aplicación de red. Se demostró que el protocolo TCP es indispensable para procesos transaccionales que requieren integridad estricta (evitando saltos o duplicidad en los turnos), mientras que UDP aporta eficiencia y ligereza para consultas informativas. Asimismo, la

implementación del módulo threading evidenció que los servidores modernos requieren arquitecturas concurrentes para no degradar la experiencia del usuario